Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Executive Summary: How a Spill Is Defined and Measured

gem5 TimingSimpleCPU · RISC-V (RV64GC) · SPEC CPU2017 · metric schema 7

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

16 September 2026

1. What is measured

A spill is detected *dynamically*, from the memory behaviour of the running program, not from compiler metadata. The detector watches every committed load and store and looks for the signature of compiler spill code: a value written to a stack slot through the stack or frame pointer, and later read back from that slot.

All counting is confined to the region of interest (ROI), marked by `m5_work_begin` and `m5_work_end` in the benchmark sources. State is cleared at ROI entry, so no pair straddles the boundary.

2. Store side

Three tests decide whether a committed store is recorded as a spill write.

Table 1. Store-side rules, applied when the store commits.

	Test	Rationale
S1	The store executes inside the ROI	Measurement window
S2	The target address lies in the stack region, `stackMin` $\leq$ addr $<$ `stackBase`	Spill slots live on the stack
S3	The address base register is `sp` (`x2`) or `s0/fp` (`x8`)	How spill code addresses the stack

A passing store is counted in `spill_store`, tagged for the cache statistics, and remembered in a map keyed by its address. A later store to that address replaces the entry, beginning a new *generation*.

3. Load side

Six tests decide whether a committed load reads back a recorded spill.

Table 2. Load-side rules, applied when the load commits.

	Test	Rationale
L1	The load executes inside the ROI	Measurement window
L2	The source address lies in the stack region	As S2
L3	A recorded store exists at exactly this address	There is a value to read back
L4	The load is later in time than that store	A spill is written before it is read
L5	Load and store use the same access width	The same value, not an overlapping one
L6	Load and matched store are both base-register relative, per S3	Both halves must look like spill code

A load passing all six is counted in `spill_load`; the first one for a generation also completes a `spill_event`. Pairs that satisfy L1–L5 but fail L6 are ordinary stack data reuse, counted separately.

4. Counters

Table 3. Reported counters and how each is derived.

Counter	Definition
`spill_store`	Write side: every store passing S1–S3, read back later or not.
`spill_load`	Read-back side: every load passing L1–L6, repeats included.
`spill_event`	One (`spill_store`, first `spill_load`) pair: one *potential spill*.
`total_spill_insts`	`spill_store` + `spill_load`.
`spill_store_unpaired`	`spill_store` − `spill_event`: written, never read back inside the ROI.
`repeat_spill_load`	`spill_load` − `spill_event`: read-backs after the first.

The two sides are decided at different moments, so two inequalities always hold: `spill_event` $\leq$ `spill_store`, since a value may be written and never read back; and `spill_load` $\geq$ `spill_event`, since a value may be read back many times.

5. What this does and does not prove

The rules establish that a stack slot was written through the stack or frame pointer and, for a `spill_event`, read back at the same width.

They do *not* establish that the register allocator ran out of registers. Three other populations pass exactly the same tests and cannot be told apart at the microarchitectural level:

- ABI-mandated callee-save save and restore in prologues and epilogues;
- ordinary scalar locals the compiler kept in memory;
- incoming arguments written to their stack home.

The figures are therefore an **upper bound** on true register-allocator spilling, and should be read as *potential* spills. The write side is the looser of the two: at store time a later read-back is unknowable, so `spill_store` need only pass S1–S3.

For an exact classification the detector also accepts a compiler-generated PC map (`spill_pc_map`), replacing tests S3 and L6 with the addresses the compiler emitted as spill and reload code.

6. Implementation

Component	Source
Rules and counters	`src/cpu/simple/spill_detector.{hh,cc}`
Base-register test, commit hook	`src/cpu/simple/timing.cc`
ROI markers, statistics boundaries	`src/sim/pseudo_inst.cc`
Per-population cache counters	`src/mem/cache/base.{hh,cc}`

The rule set is covered by 21 hand-verified RISC-V assembly test cases in `tests/spill_test/`, which also assert that the gem5 cache statistics and the detector counters agree at both cache levels.